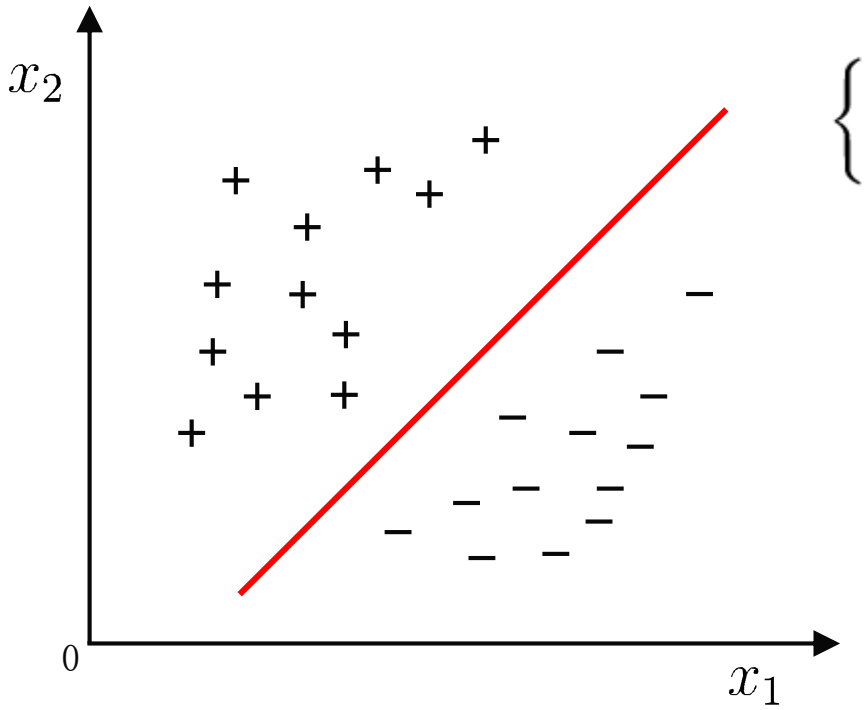
第10章 支持向量机

线性模型：在样本空间中寻找一个**超平面**，将不同类别的样本分开.

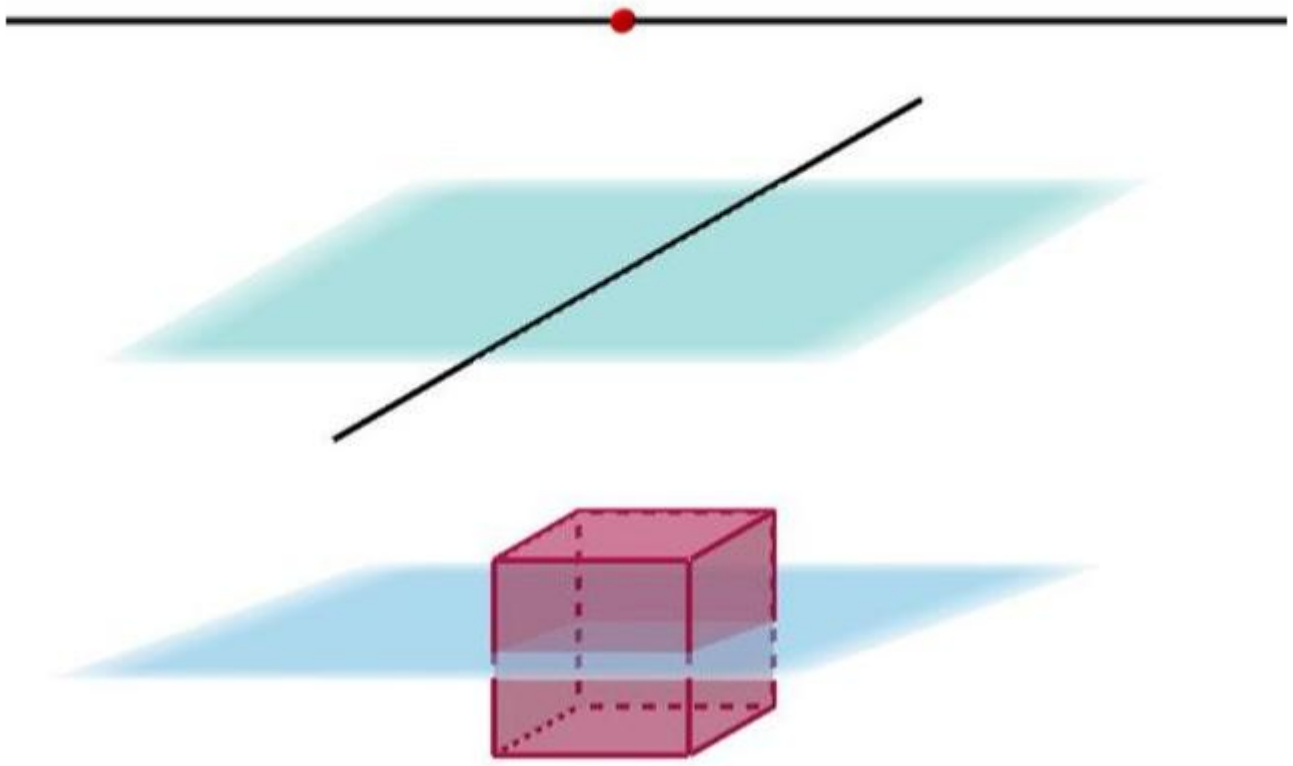


$$\begin{cases} \omega^T x + b > 0, y_i = 1 \\ \omega^T x + b < 0, y_i = -1 \end{cases}$$

$$D = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \ldots\} \qquad y_i \in \{1, -1\}$$

超平面

In geometry a **hyperplane** is a subspace of one dimension less than its ambient space.



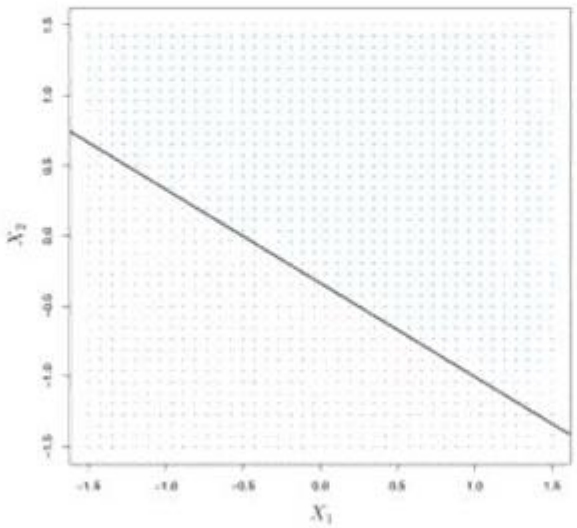
超平面的数学表达形式： $w^T x = \mathrm{b}$ $w \in R^n$ $x \in R^n$ $\Leftrightarrow$ $w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = 0$

一维空间超平面

$w_1x_1 + b = 0$

- 一维空间的超平面是一个点

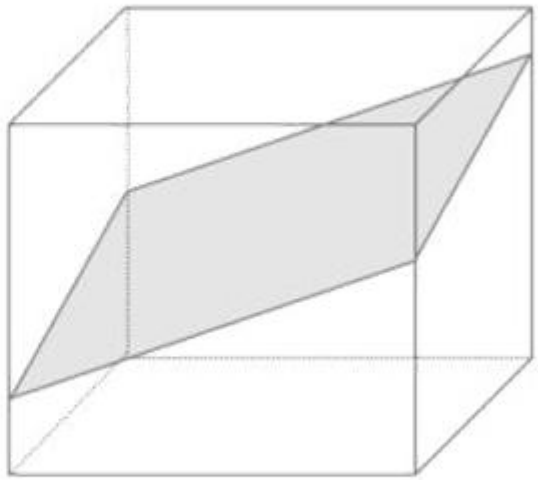
二维空间超平面



$w_1x_1 + w_2x_2 + b = 0$

- 二维空间的超平面是一条直线

三维空间超平面



$w_1x_1 + w_2x_2 + w_3x_3 + b = 0$

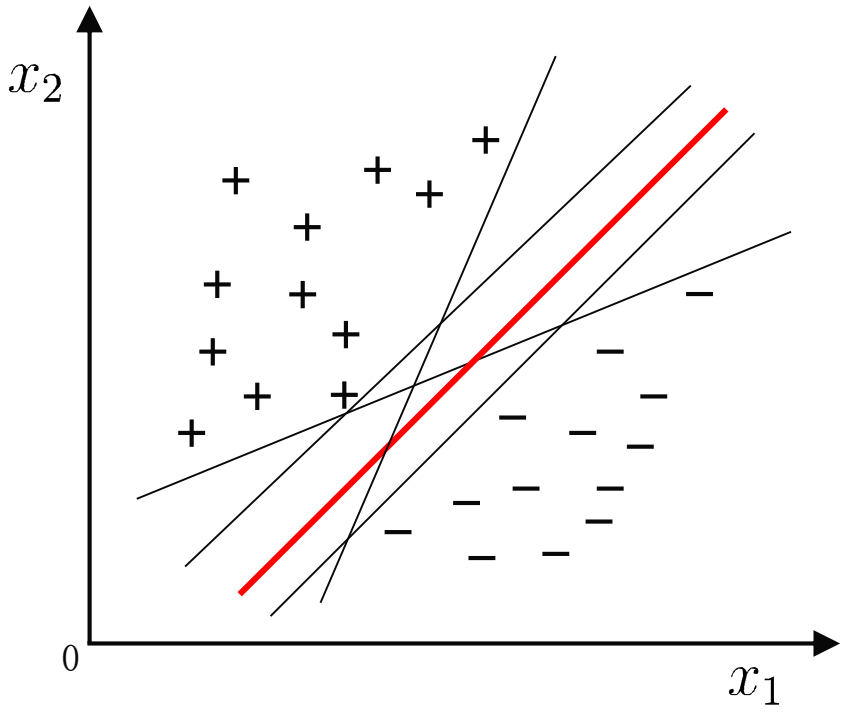
- 三维空间的超平面是一个平面

N维空间超平面

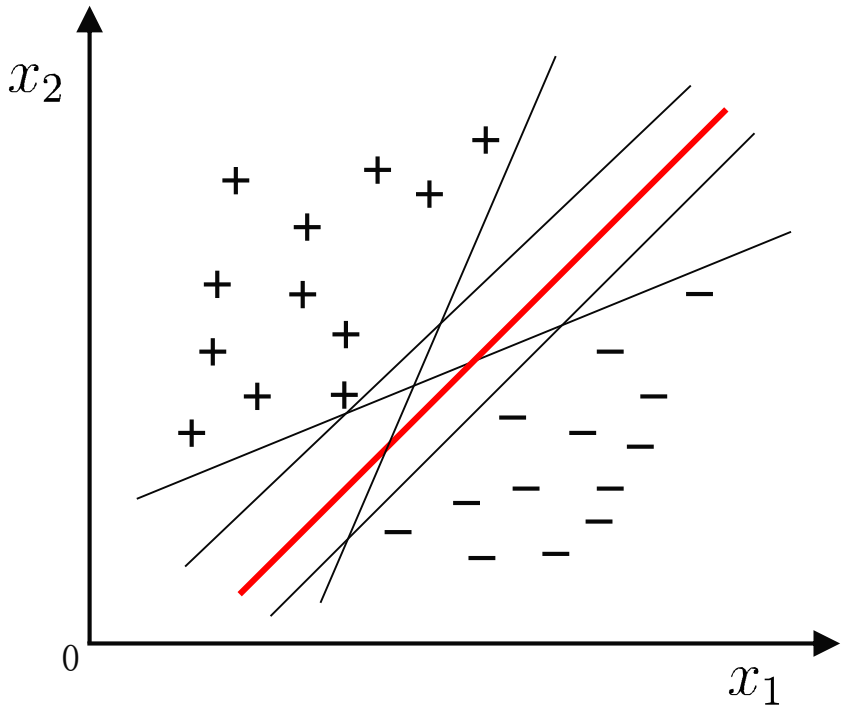
?

超过3维时，无法用几何的方式直观地来表达

-Q:将训练样本分开的超平面可能有很多，哪一个好呢?



-Q:将训练样本分开的超平面可能有很多，哪一个好呢?



$$\begin{cases} \omega^T x + b \geq 1, y_i = 1 \\ \omega^T x + b \leq -1, y_i = -1 \end{cases}$$

-A:应选择"正中间", 容忍性好, 鲁棒性高, 泛化能力最强.

二维空间点 (x, y) 到直线 $Ax + By + C = 0$ 的距离公式是：

$$\frac{|Ax + By + C|}{\sqrt{A^2 + B^2}}$$

扩展到 n 维空间后，点 $x = (x_1, x_2 \ldots x_n)$ 到 $w^T x + b = 0$ 的距离为：

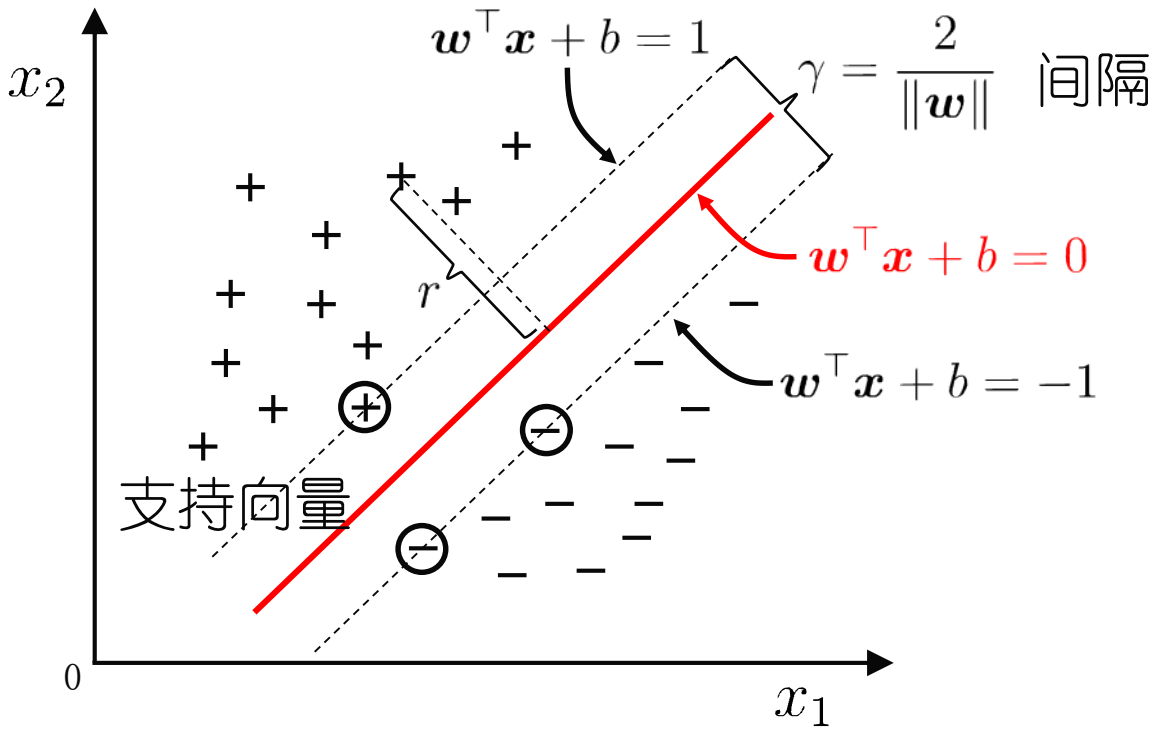
$$\frac{|w^T x + b|}{||w||}$$

其中 $||w|| = \sqrt{w_1^2 + \ldots w_n^2}$ 。

间隔与支持向量

超平面方程： $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$

$$\frac{|\omega^T x + b|}{||\omega||}$$



支持向量机基本型

最大间隔：寻找参数 $\boldsymbol{w}$ 和 b，使得 γ 最大.

$$\begin{cases} \omega^T x + b \geq 1, y_i = 1 \\ \omega^T x + b \leq -1, y_i = -1 \end{cases}$$

$$\arg\max_{\boldsymbol{w},b} \frac{2}{\|\boldsymbol{w}\|}$$

$$\text{s.t.} \quad y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \ i = 1, 2, \ldots, m.$$

$$\Downarrow$$

$$\arg\min_{\boldsymbol{w},b} \frac{1}{2}\|\boldsymbol{w}\|^2$$

$$\text{s.t.} \quad y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \ i = 1, 2, \ldots, m.$$

求解

$$\arg\min_{\boldsymbol{w},b} \ \frac{1}{2}\|\boldsymbol{w}\|^2$$
$$\text{s.t.} \ \ y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \ i = 1, 2, \ldots, m.$$

拉格朗日乘子法

第一步：引入拉格朗日乘子 $\alpha_i \geq 0$ 得到拉格朗日函数

$$L(\boldsymbol{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}||\boldsymbol{w}||^2 + \sum_{i=1}^{m} \alpha_i (1 - y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b))$$

对偶问题

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j$$

最终模型： $f(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x} + b = \boxed{\sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x}} + b$

KKT条件 $\begin{cases} \alpha_i \geq 0, \\ y_i f(\boldsymbol{x}_i) \geq 1, \\ \alpha_i (y_i f(\boldsymbol{x}_i) - 1) = 0. \end{cases}$

支持向量机解的稀疏性：训练完成后，大部分的训练样本都不需保留，最终模型仅与支持向量有关.

线性SVM的语法

导入包含分类方法的类：

```
from sklearn.svm import LinearSVC
```

创建该类的一个对象：

```
linSVC = LinearSVC( )
```

拟合训练数据，并预测测试数据：

```
linSVC = linSVC.fit(X_train, y_train)
y_predict = linSVC.predict(X_test)
```

线性SVM的语法

导入包含分类方法的类：

```
from sklearn.svm import LinearSVC
```

创建该类的一个对象：

```
linSVC = LinearSVC(penalty='l2', C=10.0)
```

正则化参数

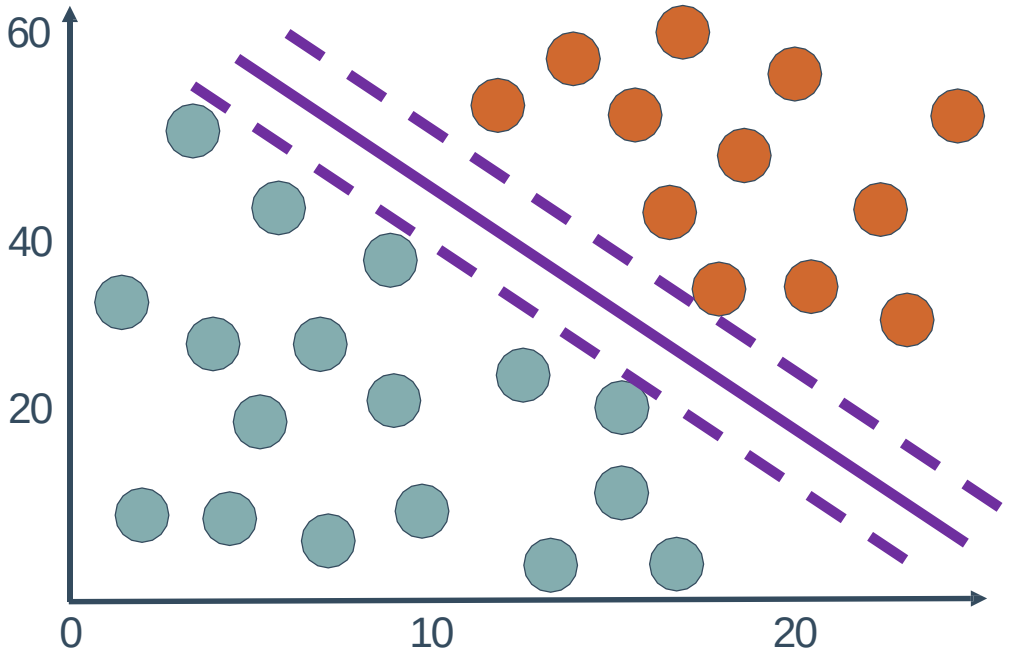
拟合训练数据，并预测测试数据：

```
linSVC = linSVC.fit(X_train, y_train)
y_predict = linSVC.predict(X_test)
```

http://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html

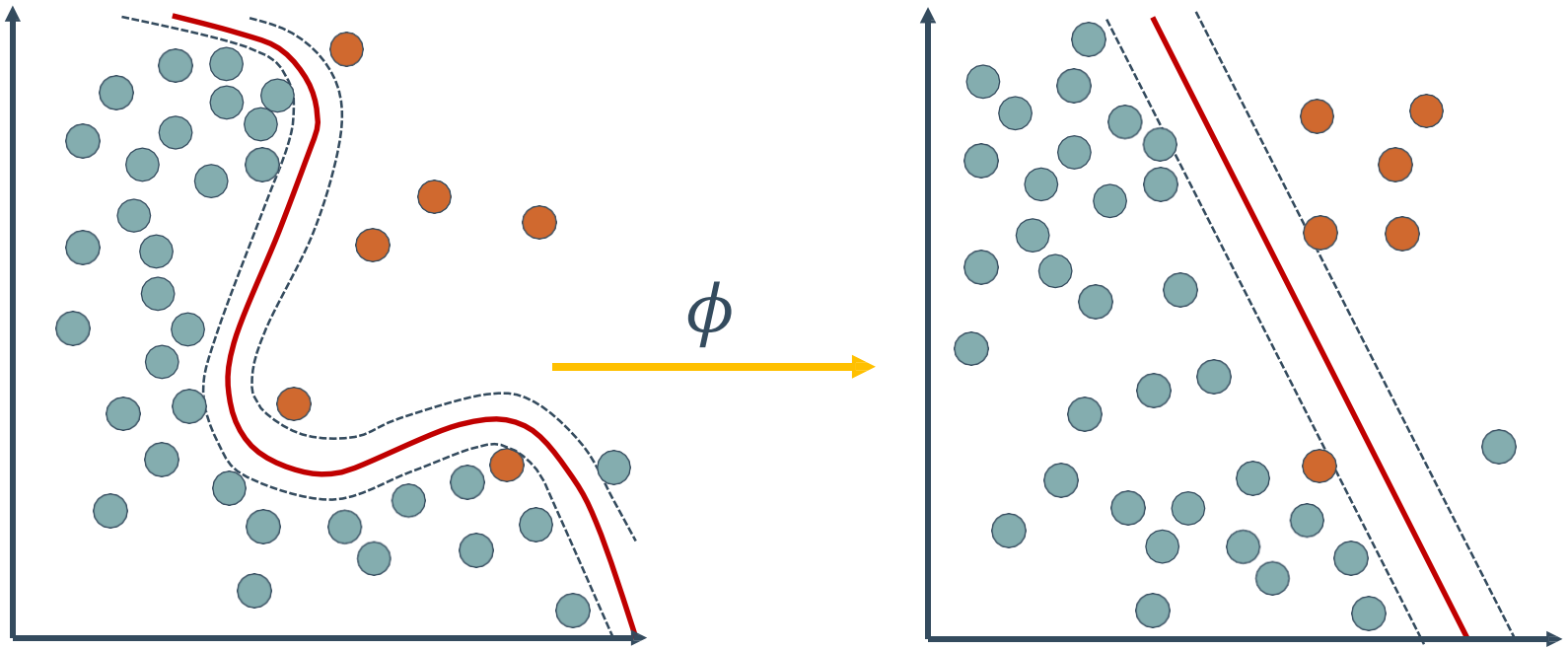
核函数

用SVM分类



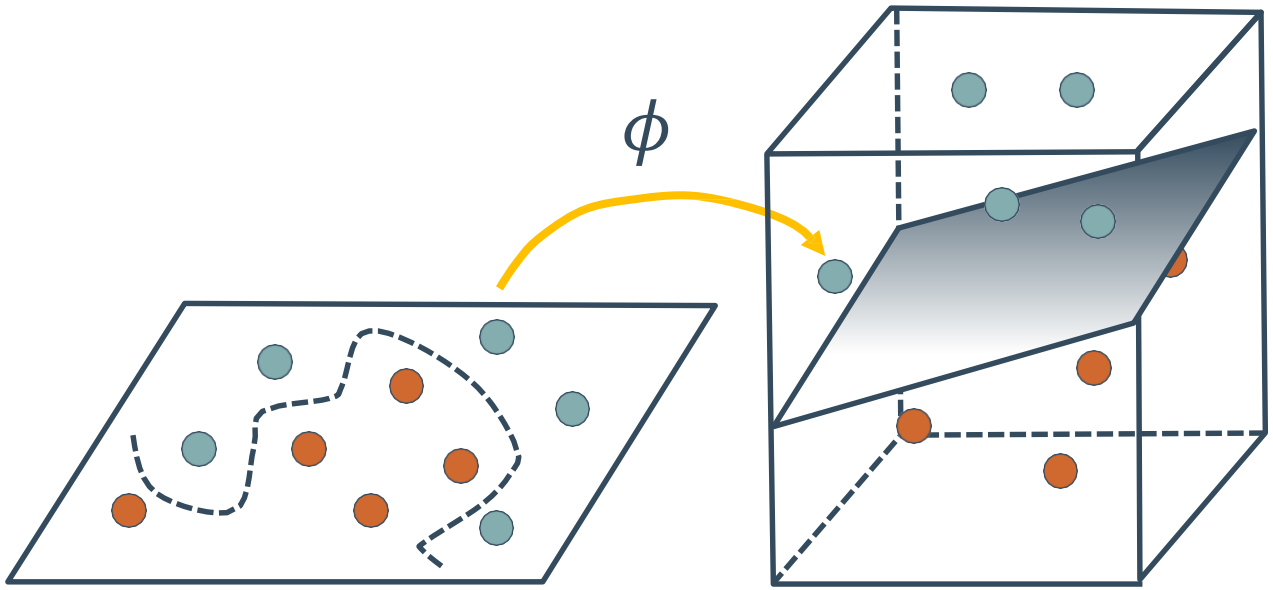
非线性判定边界

非线性数据在高维空间可能被转换为线性的

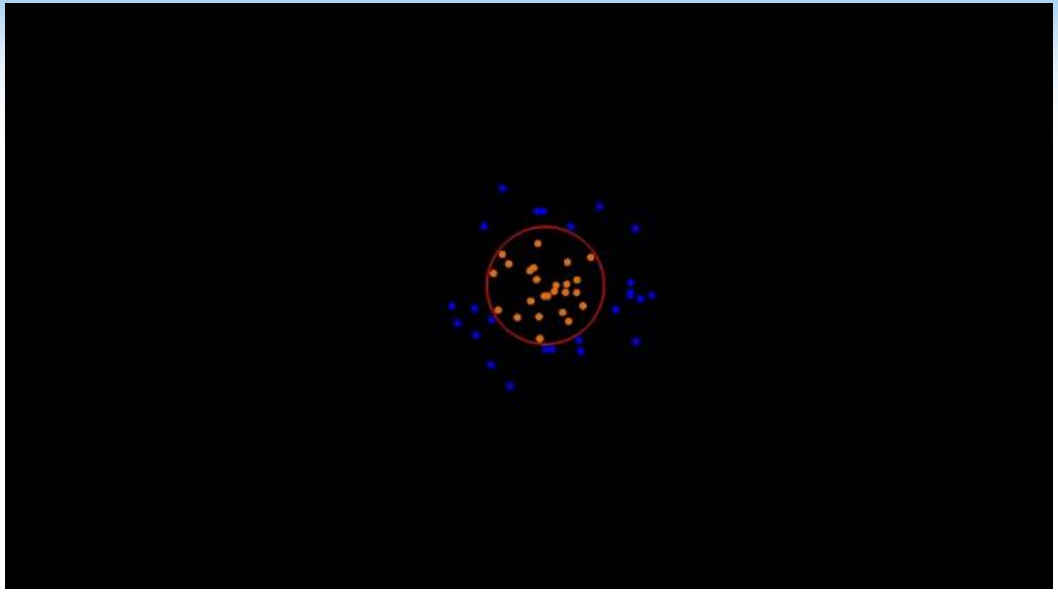


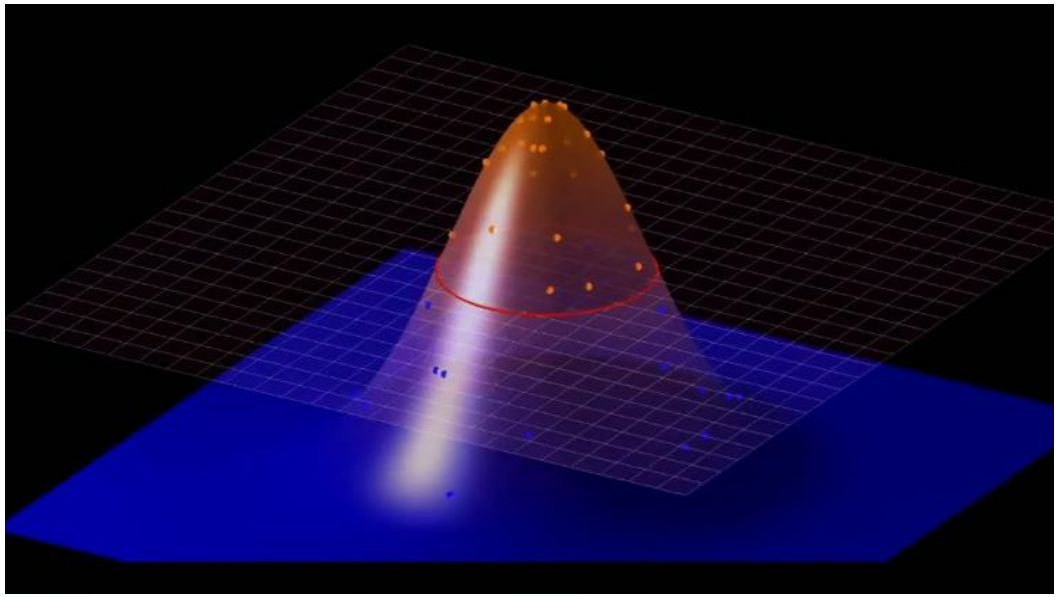
高维映射

把数据转换为线性可分的



如果原始空间是有限维，即属性数有限，那么一定存在一个高维特征空间使样本可分。





核支持向量机

设样本 $\boldsymbol{x}$ 映射后的向量为 $\phi(\boldsymbol{x})$，划分超平面 $f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b$

原始问题

$$\min_{\boldsymbol{w},b} \ \frac{1}{2}\|\boldsymbol{w}\|^2$$

$$\text{s.t.} \ \ y_i(\boldsymbol{w}^\top \phi(\boldsymbol{x}_i) + b) \geq 1, \ \ i = 1, 2, \ldots, m.$$

对偶问题

$$\max_{\boldsymbol{\alpha}} \ \sum_{i=1}^{m} \alpha_i - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \boxed{\phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)}$$

只以内积的形式出现

$$\text{s.t.} \ \ \sum_{i=1}^{m} \alpha_i y_i = 0, \ \ \alpha_i \geq 0, \ \ i = 1, 2, \ldots, m.$$

预测

$$f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b = \sum_{i=1}^{m} \alpha_i y_i \boxed{\phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x})} + b$$

核函数

基本想法：不显式地设计核映射，而是设计**核函数**.

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)$$

Mercer定理(充分非必要)：只要一个对称函数所对应的核矩阵**半正定**，则它就能作为核函数来使用.

常用核函数：

名称	表达式	参数
线性核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^\top \boldsymbol{x}_j$	
多项式核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^\top \boldsymbol{x}_j)^d$	$d \geq 1$为多项式的次数
高斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert \boldsymbol{x}_i - \boldsymbol{x}_j \rVert^2}{2\delta^2}\right)$	$\delta > 0$为高斯核的带宽(width)
拉普拉斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert \boldsymbol{x}_i - \boldsymbol{x}_j \rVert}{\delta}\right)$	$\delta > 0$
Sigmoid核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \tanh(\beta \boldsymbol{x}_i^\top \boldsymbol{x}_j + \theta)$	tanh为双曲正切函数, $\beta > 0$, $\theta < 0$

使用核函数的SVM的语法

导入包含分类方法的类：

```
from sklearn.svm import SVC
```

创建该类的一个对象：

```
rbfSVC = SVC(kernel='rbf', gamma=1.0, C=10.0)
```

拟合训练数据，并预测：

```
rbfSVC = rbfSVC.fit(X_train, y_train)
y_predict = rbfSVC.predict(X_test)
```

使用核函数的SVM的语法

导入包含分类方法的类：

```
from sklearn.svm import SVC
```

创建该类的一个对象：

```
rbfSVC = SVC(kernel='rbf', gamma=1.0, C=10.0)
```

设置核函数，及其相应参数(gamma).

$$\gamma = \frac{1}{2\sigma^2}$$

拟合训练数据，并预测：

```
rbfSVC = rbfSVC.fit(X_train, y_train)
y_predict = rbfSVC.predict(X_test)
```

使用核函数的SVM的语法

导入包含分类方法的类：

```
from sklearn.svm import SVC
```

创建该类的一个对象：

```
rbfSVC = SVC(kernel='rbf', gamma=1.0, C=10.0)
```

"C" 是错误项的惩罚力度

拟合训练数据，并预测：

```
rbfSVC = rbfSVC.fit(X_train, y_train)
y_predict = rbfSVC.predict(X_test)
```

http://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

使用核函数的SVM的语法

导入包含分类方法的类：

```
from sklearn.svm import SVC
```

创建该类的一个对象：

```
rbfSVC = SVC(kernel='rbf', gamma=1.0, C=10.0)
```

拟合训练数据，并预测：

```
rbfSVC = rbfSVC.fit(X_train, y_train)
y_predict = rbfSVC.predict(X_test)
```

用交叉验证调节核函数及其参数

class **sklearn.svm.SVC**(*C=1.0, kernel='rbf', degree=3, gamma='auto', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', random_state=None*)

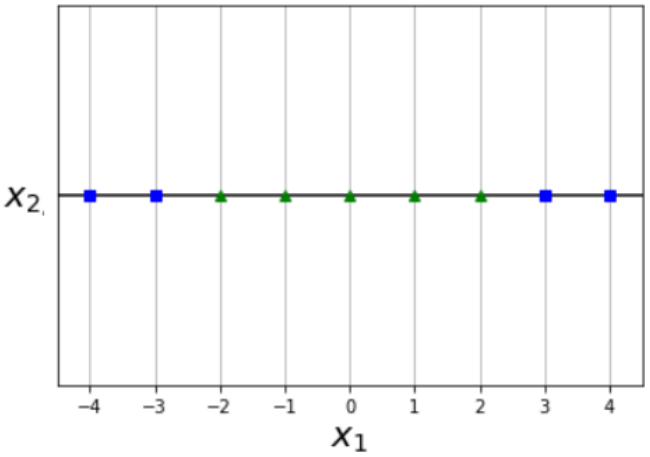
- kernel:核函数类型，str类型，默认为‘rbf’。可选参数为：
  - ‘linear’：线性核函数
  - ‘poly’：多项式核函数
  - ‘rbf’：径向基核函数/高斯核
  - ‘sigmod’：sigmod核函数
  - ‘precomputed’：核矩阵。precomputed表示自己提前计算好核函数矩阵，这时候算法内部就不再用核函数去计算核矩阵，而是直接用该n*n的核矩阵。

- **线性核函数：**这是最简单的核函数，它直接计算两个输入特征向量的内积。
  - 优点：简单高效，结果易解释，总能生成一个最简洁的线性分割超平面
  - 缺点：只适用线性可分的数据集

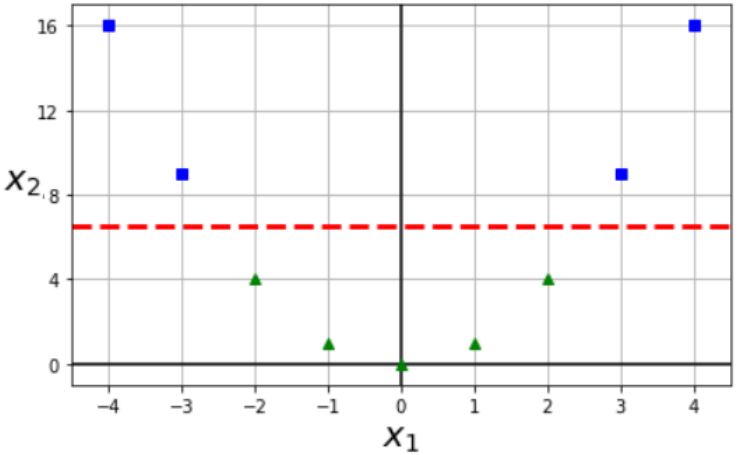
$$K(x,z) = x \cdot z$$

- **多项式核函数：**通过多项式来作为特征映射函数
  - 优点：可以拟合出复杂的分割超平面。
  - 缺点：参数太多。有γ, r, d三个参数要选择，选择起来比较困难；另外多项式的阶数不宜太高否则会给模型求解带来困难。

$$K(x,z) = (\gamma x \cdot z + r)^d$$



低维线性不可分数据



映射后的高维线性可分数据

- **高斯核函数：**
  - 优点：可以把特征映射到无限多维，并且没有多项式计算那么困难，参数也比较好选择。
  - 缺点：不容易解释，计算速度比较慢，容易过拟合。

$$K\left(x^{(i)}, x^{(j)}\right) = \exp\left(-\frac{\left(x^{(i)} - x^{(j)}\right)^2}{2\sigma^2}\right)$$

$$\text{高斯预测函数} = \sum_{i=1}^{m} a_i y^{(i)} K\left(x^{(i)}, x\right) + b$$

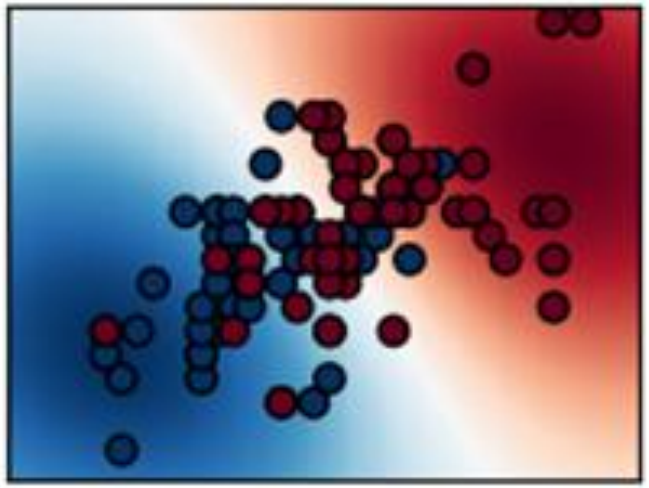
预测函数是中心点在支持向量处的高斯函数的线性组合。因此，高斯核函数也称为 **RBF核函数**，即反钟形函数的线性组合。

- degree:多项式核函数的阶数，int类型，可选参数，默认为3。这个参数只对多项式核函数有用，是指多项式核函数的阶数n，如果是其他核函数，则会自动忽略该参数。
- coef0: 核函数常数值(y=kx+b中的b值)，只有‘poly’和‘sigmoid’核函数有，float类型，可选参数，默认为0.0。

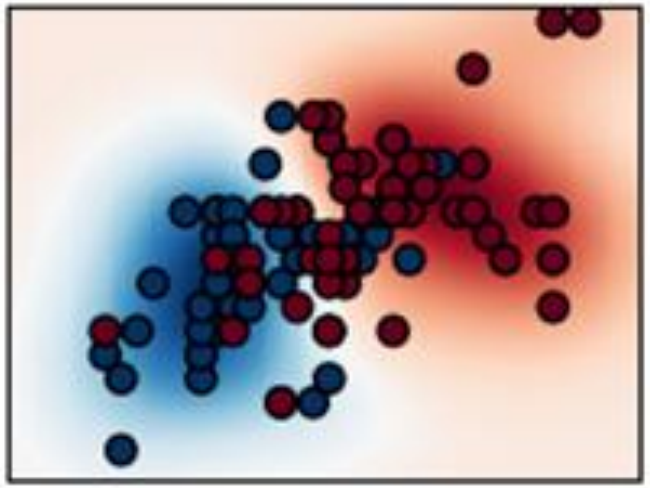
- gamma:核函数系数，float类型，可选参数，默认为auto。只对’rbf’,’poly’ ,’sigmod’有效。如果gamma为auto，代表其值为样本特征数的倒数，即1/n_features。（即核函数的带宽，超圆的半径）。gamma越大，σ越小，使得高斯分布又高又瘦，造成模型只能作用于支持向量附近，可能导致过拟合；反之，gamma越小，σ越大，高斯分布会过于平滑，在训练集上分类效果不佳，可能导致欠拟合。

径向基核函数参数gamma

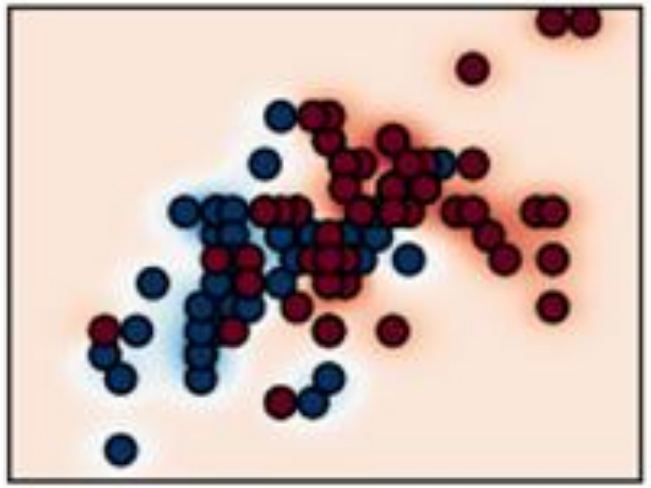
gamma=10^-1



gamma=10^0



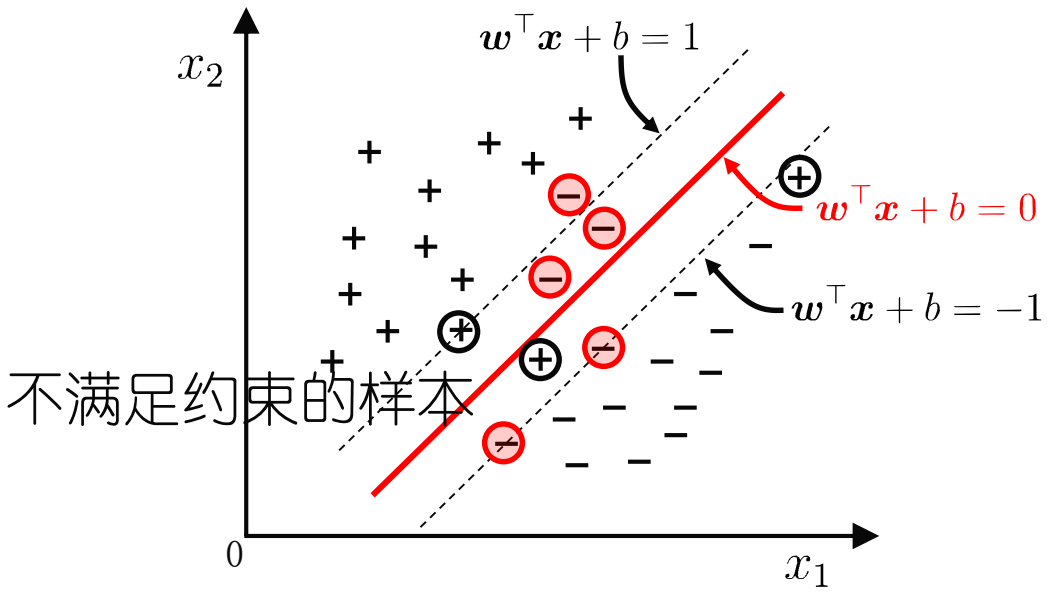
gamma=10^1



软间隔

-Q:现实中，很难确定合适的核函数使得训练样本在特征空间中线性可分；同时一个线性可分的结果也很难断定是否是由过拟合造成的.

-A:引入“**软间隔**”的概念，允许支持向量机在一些样本上不满足约束.



0/1损失函数

基本想法：最大化间隔的同时，让不满足约束的样本应尽可能少.

$$\min_{\boldsymbol{w},b} \ \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{m} l_{0/1}\left(y_i(\boldsymbol{w}^\top\phi(\boldsymbol{x}_i)+b)-1\right)$$

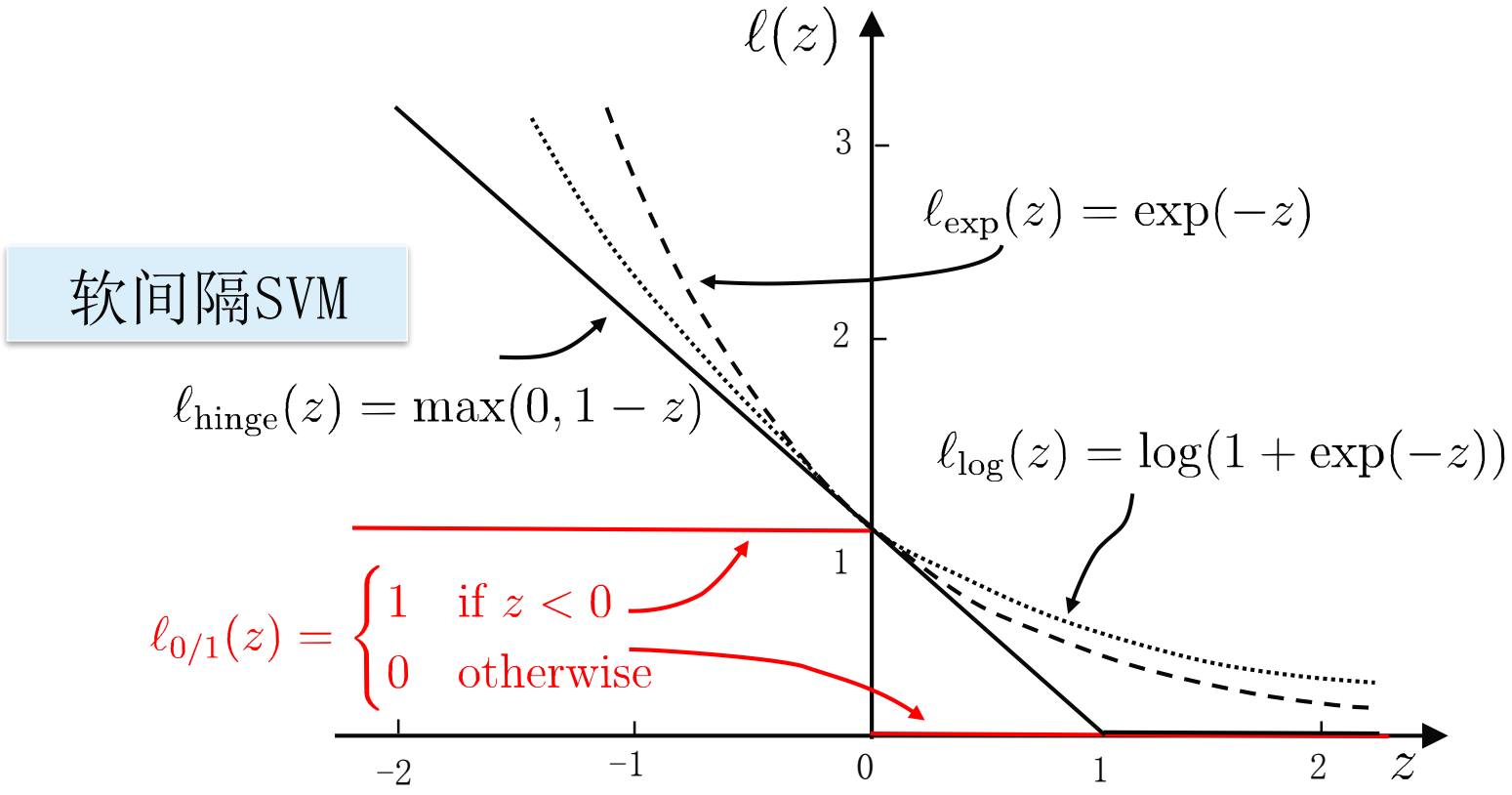
其中 $l_{0/1}$ 是”0/1损失函数”

$$l_{0/1} = \begin{cases} 1 & z < 0 \\ 0 & otherwise \end{cases}$$

正则化常数 C > 0, 如果C → ∞，则等价于要求所有的样本点都分类正确，否则就允许一部分极少的样本分类错误

存在的问题：0/1损失函数非凸、非连续，不易优化！

替代损失



替代损失函数数学性质较好，一般是0/1损失函数的上界

软间隔支持向量机

原始问题 $$\min_{\boldsymbol{w},b} \ \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{m}\max\left(0, 1 - y_i(\boldsymbol{w}^\top\phi(\boldsymbol{x}_i) + b)\right)$$

对偶问题

$$\min_{\boldsymbol{\alpha}} \ \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j \phi(\boldsymbol{x}_i)^\top\phi(\boldsymbol{x}_j) - \sum_{i=1}^{m}\alpha_i$$

$$\text{s.t.} \ \sum_{i=1}^{m}\alpha_i y_i = 0, \ 0 \le \alpha_i \le C, \ i = 1, 2, \ldots, m.$$

根据KKT条件可推得最终模型仅与支持向量有关，也即hinge损失函数依然保持了支持向量机解的**稀疏性**.

原始问题 $\min\limits_{\mathbf{w},b} \frac{1}{2}\|w\|^2 + C\sum\limits_{i=1}^{m}\max\left(0, 1-y_i\left(w\cdot x_i + b\right)\right)$

引入“松弛变量(Slack Variable)” $\xi_i \geq 0$，

$$\min_{\mathbf{w},b,\xi_i} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\xi_i$$

$$s.t.\quad y_i\left(w\cdot x_i + b\right) \geq 1-\xi_i$$ ---软间隔SVM，

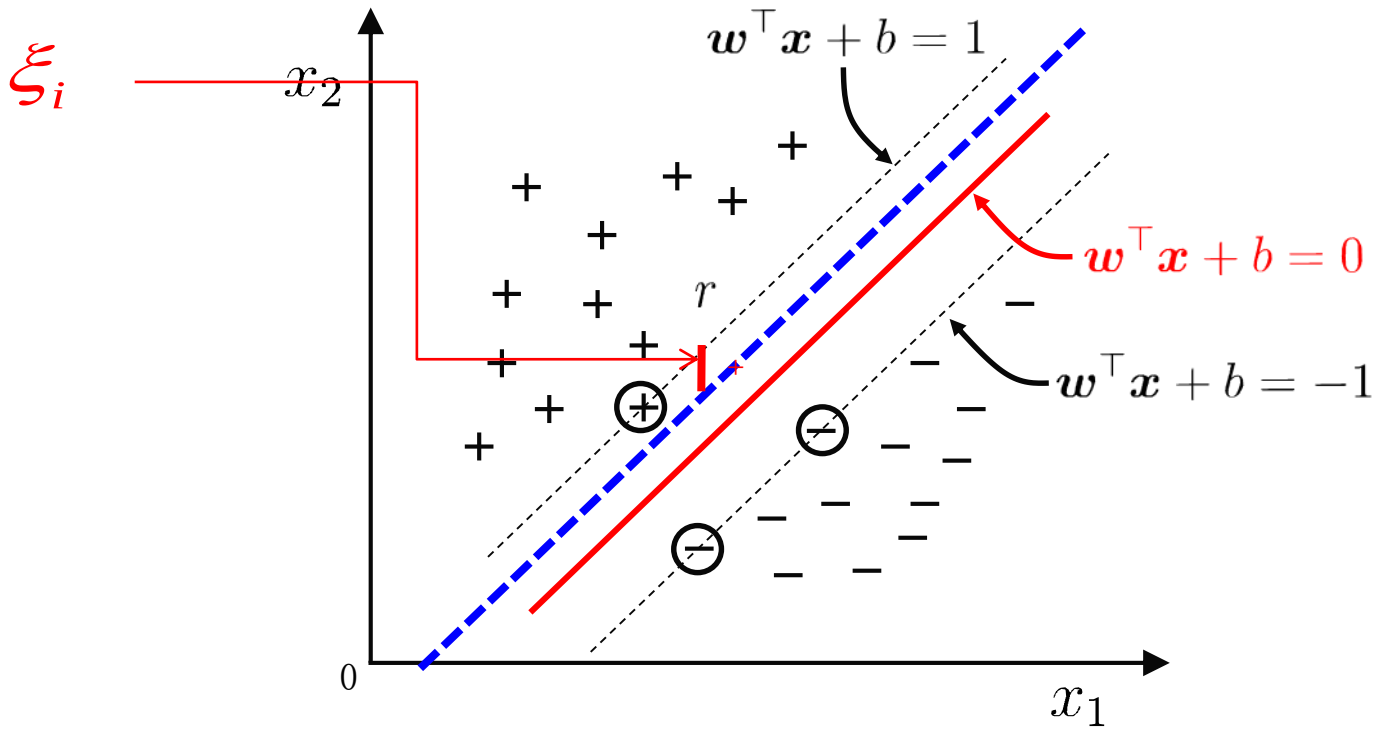
又称C-SVC

$$\xi_i \geq 0, i = 1, 2, \ldots, m$$

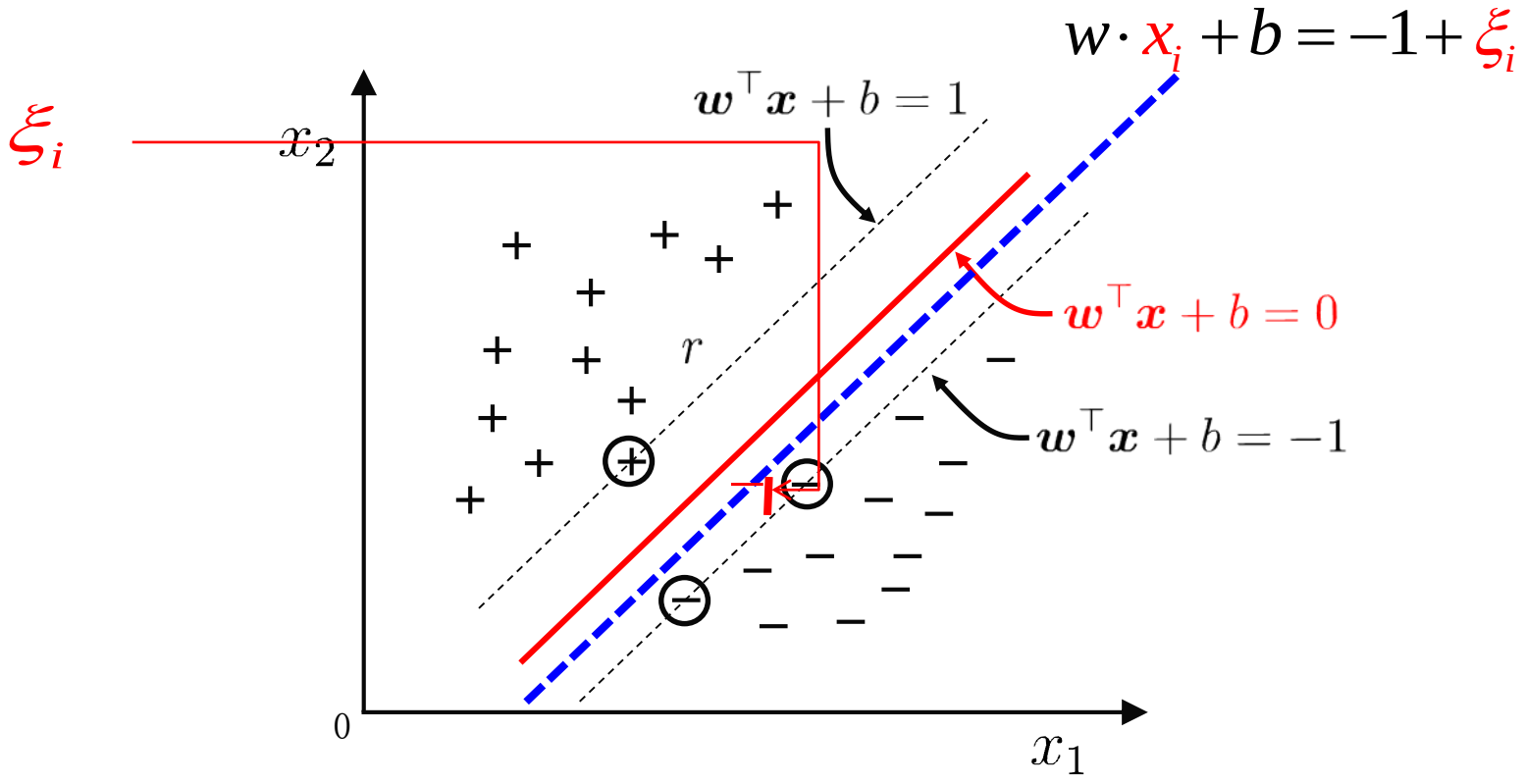
注：每一个样本都对应一个松弛变量ξ_i，用以表征该样本不满足约束 $y_i f\left(x_i\right) \geq 1$的程度。

超平面方程： $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$

$$w \cdot x_i + b = 1 - \xi_i$$



超平面方程： $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$



正则化

支持向量机学习模型的更一般形式

$$\min_f \ \Omega(f) + C\sum_{i=1}^{m} l(f(\boldsymbol{x}_i), y_i)$$

结构风险，描述模型的某些性质

经验风险，描述模型与训练数据的契合程度

通过替换上面两个部分，可以得到许多其他学习模型

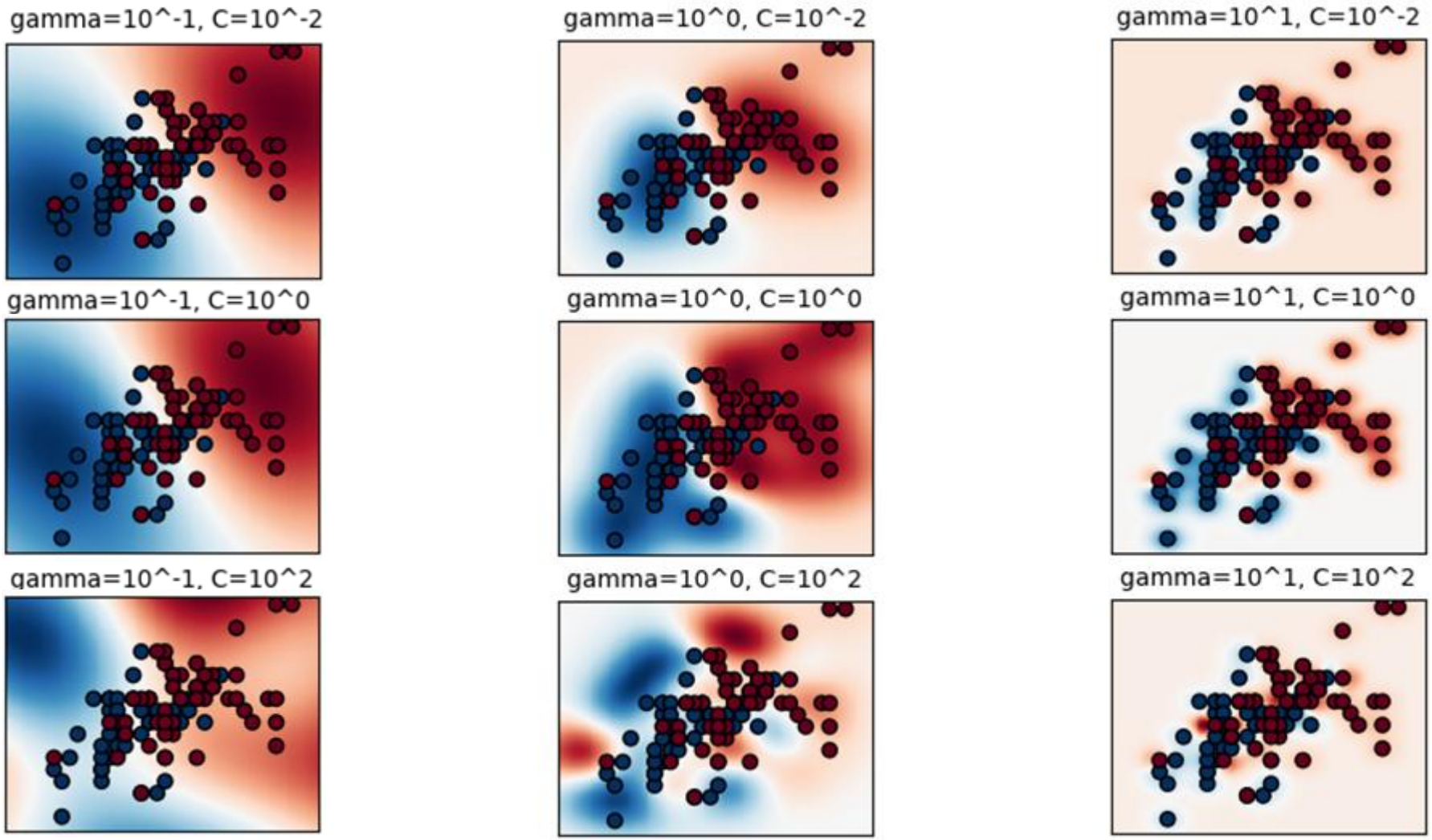
- 对数几率回归(Logistic Regression)
- LASSO
- ……

$$\min_w C\sum_{i=1}^{n}\left(-y_i\log(\hat{p}(X_i)) - (1-y_i)\log(1-\hat{p}(X_i))\right) + r(w).$$

class **sklearn.svm.SVC**(*C=1.0, kernel='rbf', degree=3, gamma='auto', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', random_state=None*)

- C：惩罚项，float类型，可选参数，默认为1.0。C越大，即对分错样本的惩罚程度越大，因此在训练样本中准确率越高，但是泛化能力降低，也就是对测试数据的分类准确率降低。相反，减小C的话，容许训练样本中有一些误分类错误样本，泛化能力强。一般取值为10的n次幂，如10的-5次幂，10的-4次幂……10的0次幂，10，1000,1000，在python中可以使用pow（10，n） 。

径向基核函数参数gamma和C



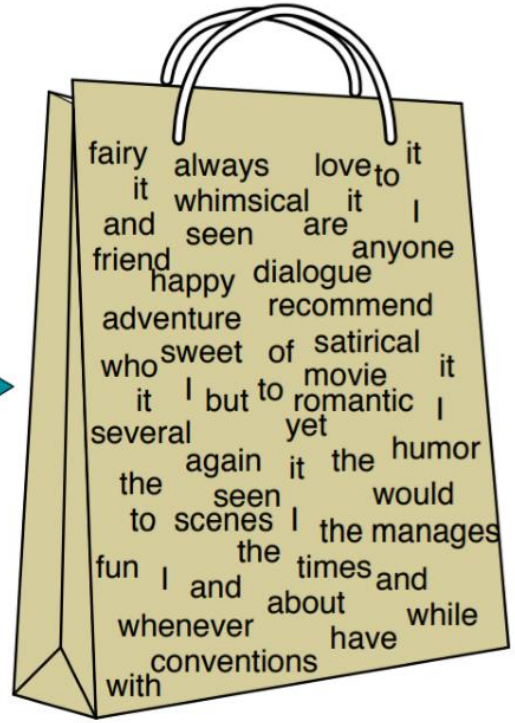
- tol：停止训练的误差精度，float类型，可选参数，默认是0.0001。
- max_iter：最大迭代次数，int类型，默认是-1，即没有限制。这个是硬限制，它的优先级要高于tol参数，不论训练的标准和精度达到要求没有，都要停止训练。

- decision_function_shape ：决策函数类型，可选参数’ovo’和’ovr’，默认为’ovr’。原始的SVM只适用于二分类问题，如果要将其扩展到多类分类，就要采取一定的融合策略， ‘ovo’ 一对一，为one v one，即将类别两两之间进行划分，用二分类的方法模拟多分类的结果，决策所使用的返回的是（样本数，类别数*(类别数-1)/2）； ‘ovr’ 一对多，为one v rest，即一个类别与其他类别进行划分，返回的是(样本数，类别数）。

文本分类—垃圾邮件识别

词袋模型（Bag of words， BoW)

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

John likes to watch movies. Mary likes too.
John also likes to watch football games.

{"John": 1, "likes": 2,"to": 3, "watch": 4, "movies": 5,"also": 6, "football": 7, "games": 8,"Mary": 9, "too": 10}

[1, 2, 1, 1, 1, 0, 0, 0, 1, 1]
[1, 1, 1, 1, 0, 1, 1, 1, 0, 0]

用CountVectorizer统计词频

CountVectorizer 不仅提供了一个简单的方法为一系列文本文件进行分词操作，从而用这些已有的词建立一个词汇表，同时还能用这个词汇表对新文档编码。

使用步骤如下：

1. 创建CountVectorizer 类的一个实例
2. 调用fit() 函数从一个或多个文件中学习一个词汇表
3. 调用transform() 函数将一个文件编码成一个向量

返回一个与词汇表同长的向量，向量中每个位置对应词汇表该位置上的词在当前文件中出现的次数。

TF-IDF（term frequency - inverse document frequency）

单个文本的样本称为**文档**(document)，而所有文档的集合称为**语料库**(corpus)。

- 淘宝购物后的评语，每一条评语称为**文档**，所有的评语称为**语料库**；
- 一篇论文可称为**文档**，所有的论文称为**语料库**。

TF 词频计算

r_j 代表文档， w_i 代表单词

r_1	w_1	$\boxed{w_2}$	w_3	$\boxed{w_2}$	w_5	w_6			→ 共6个单词
r_2	w_1	$\boxed{w_3}$	$\boxed{w_3}$	w_4	w_5	w_6	w_4	w_7	→ 共8个单词
r_3	…			…			…		
⋮									
r_n	…			…			…		

TF的通用公式为： $TF(w_i, r_j) = \frac{\text{当前文档里单词}w_i\text{的出现次数}}{\text{当前文档单词总数}}$

- 在文档 r_1 中 w_2 的词频为： $TF(w_2, r_1) = \frac{2}{6} = \frac{1}{3}$
- 在文档 r_2 中 w_3 的词频为： $TF(w_3, r_2) = \frac{2}{8} = \frac{1}{4}$

IDF 逆文本频率指数计算

r_j 代表文档， w_i 代表单词

r_1	w_1	w_2	$\boxed{w_3}$	w_2	w_5	w_6		
r_2	w_1	$\boxed{w_3}$	$\boxed{w_3}$	w_4	w_5	w_6	w_4	w_7
r_3	$\cdots$			$\cdots$	$\boxed{w_3}$		$\cdots$	
$\vdots$								
r_{10}	$\cdots$		$\boxed{w_3}$	$\cdots$			$\cdots$	
$\vdots$								
r_n	$\cdots$			$\cdots$			$\cdots$	

IDF的通用公式为： $IDF(w_i, D_c) = log\left(\frac{\text{总文档数}}{\text{包含}w_i\text{的文档数}}\right)$,语料库 $D_c = \{r_1, r_2, \cdots, r_n\}$

在所有文档中，包含单词w_3 的有 $\{r_1, r_2, r_3, r_{10}\}$ 四个文档，所以关于 单词w_3 逆文本频率 $IDF(w_3, D_c) = log\left(\frac{n}{4}\right)$ ，一般 n 是固定的比较大，如果分母非常小，所以需要 log 来进行量纲问题处理。

TF-IDF

$$TF-IDF = TF(w_i, r_j) * IDF(w_i, D_c)$$

如果某个单词w_i 在某个文档 r_j 中的频数很大,那么**TF**就会很大 ； 同时w_i只在语料库 D_c 少量文档中出现,那么**IDF**就会很大，那么 **TF-IDF**就会更大。

TF-IDF 就是更加 重视语料库中较罕见的词 和 更加 重视单个文档中较常见的词。